

Network Slicing in SDN con Mininet e Ryu

Network and Cloud Infrastructures — a.a. 2025/2026 — Prof. Giorgio Ventre

Malafronte Alessio (DE9000200) | Persico Carmine (DE9000135)

Repository: <https://github.com/NCIs-unina/ncis-project-work-2026-malafronte-persico>

1 Obiettivo, ambiente e struttura del codice

Il progetto implementa il *network slicing* in ambiente SDN tramite un unico controller progettato per operare in due modalità distinte: **topology** e **service**.

Queste due logiche non possono essere attive contemporaneamente perché hanno scopi opposti:

- Il **topology slicing** punta all'isolamento: permette la comunicazione *solo* alle coppie di host autorizzate, assegnando a ciascuna un percorso fisico separato.
- Il **service slicing** punta alla differenziazione: richiede che *tutti* gli host possano parlarsi, ma instrada i pacchetti su percorsi diversi in base al tipo di servizio trasportato.

Per gestire entrambi gli scenari senza duplicare il codice, la modalità operativa desiderata viene semplicemente selezionata all'avvio tramite una variabile d'ambiente:

```
ryu-manager slicing_controller.py (topology, default)
SLICING_MODE=service ryu-manager slicing_controller.py (service)
```

2 Topologia

Quattro switch e quattro host, con due percorsi paralleli fra le due zone della rete. I nodi **s1** e **s4** sono switch di accesso, **s2** e **s3** di transito.

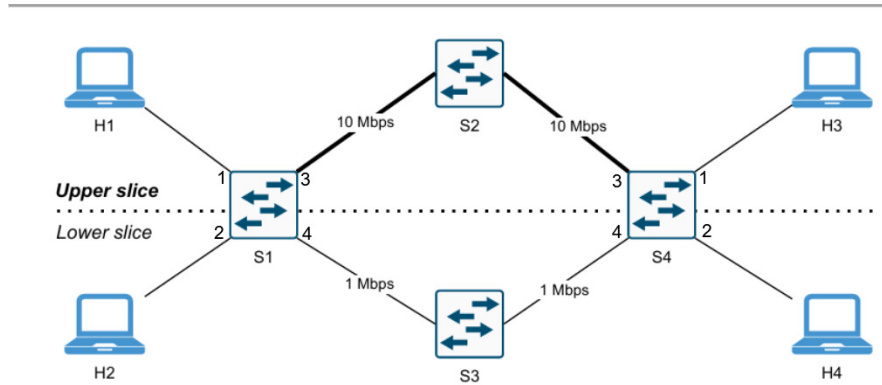


Figura 1: Topologia: due percorsi paralleli fra s1 e s4, con porte, banda e ritardo

Collegamento	Banda	Ritardo	Ruolo
host ↔ switch di accesso	non limitata	0,1 ms	non deve essere il collo di bottiglia
s1-s2-s4 (slice UPPER)	10 Mbps	2 ms	slice premium
s1-s3-s4 (slice LOWER)	1 Mbps	5 ms	slice best effort

I numeri di porta sono fissati esplicitamente in `addLink` e non lasciati alla numerazione implicita di Mininet, perché il controller li usa come costanti: riordinando le chiamate la numerazione cambierebbe e l'inoltro avverrebbe su porte errate *senza generare alcun errore*. Su entrambi gli switch di accesso vale la convenzione **porta 3 = upper**, **porta 4 = lower**. I link di accesso non hanno limite di banda proprio per non introdurre un secondo collo di bottiglia. I MAC sono statici perché il controller decide su `eth_src` ed `eth_dst`; IPv6 viene disattivato sui nodi, altrimenti il traffico multicast automatico di Linux genererebbe `packet_in` continui con sorgenti sconosciute.

3 Architettura del controller

Il controller è interamente **dichiarativo**: i percorsi non vengono appresi, sono definiti in tabelle consultate a ogni `packet_in`. La struttura centrale della modalità `topology` associa a ogni slice, per ogni switch, la porta d'uscita corrispondente a ciascuna porta d'ingresso:

```
SLICE_PATH = {
  'UPPER': {1: {1: 3, 3: 1},      # s1: h1 <-> s2
            2: {1: 2, 2: 1},      # s2: s1 <-> s4
            4: {1: 3, 3: 1}},     # s4: h3 <-> s2
  'LOWER': {1: {2: 4, 4: 2}, 3: {1: 2, 2: 1}, 4: {2: 4, 4: 2}},
}
```

La mappa e' simmetrica, quindi una sola definizione copre entrambi i versi di marcia. Accanto ad essa, SLICE_OF_MAC esprime l'appartenenza degli host agli slice, HOST_LOCATION la loro posizione, e BCAST_TREE l'albero di inoltro del traffico broadcast in modalita' service.

La gestione di una topologia ad anello chiuso impone di superare l'approccio basato sul MAC learning. Poiché la fase di apprendimento si basa sull'inoltro in *flooding* per i MAC sconosciuti, l'esistenza del loop fisico genererebbe un *broadcast storm* problematico per il controller.

La soluzione per evitare l'anello consiste nell'imporre gli instradamenti in modo dichiarativo. Definendo a priori le regole di inoltro, il problema del ciclo chiuso viene aggirato per costruzione e l'ARP naviga unicamente sui percorsi sicuri dello *slice*. L'isolamento del traffico è comunque garantito dal controllo formale sui MAC (tramite match su `eth_src` ed `eth_dst`). Questa implementazione evidenzia un interessante aspetto architetturale: la logica di instradamento basata sugli *slice* previene intrinsecamente i loop logici, andando a sostituire in toto l'intervento dello Spanning Tree Protocol (STP)..

Priorita' e scarto. OpenFlow risolve i match multipli scegliendo la regola a priorita' maggiore: 20 per il traffico video (match su livello 2, 3 e 4), 10 per l'inoltro ordinario, 5 per lo scarto, 0 per la table-miss che invia il pacchetto al controller. Lo scarto non usa un'azione dedicata: una flow entry *priva di istruzioni* fa scartare il pacchetto, e nel codice si ottiene passando una lista di azioni vuota a `_add_flow`.

4 Topology slicing: risultati

Configurazione: h1 comunica solo con h3 attraverso lo slice upper, h2 solo con h4 attraverso il lower.

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s4-eth1
h4 h4-eth0:s4-eth2
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:s2-eth1 s1-eth4:s3-eth1
s2 lo: s2-eth1:s1-eth3 s2-eth2:s4-eth3
s3 lo: s3-eth1:s1-eth4 s3-eth2:s4-eth4
s4 lo: s4-eth1:h3-eth0 s4-eth2:h4-eth0 s4-eth3:s2-eth2 s4-eth4:s3-eth2
c0
mininet> link
invalid number of args: link end1 end2 [up down]
mininet> links
h1-eth0<->s1-eth1 (OK OK)
h2-eth0<->s1-eth2 (OK OK)
h3-eth0<->s4-eth1 (OK OK)
h4-eth0<->s4-eth2 (OK OK)
s1-eth3<->s2-eth1 (OK OK)
s2-eth2<->s4-eth3 (OK OK)
s1-eth4<->s3-eth1 (OK OK)
s3-eth2<->s4-eth4 (OK OK)
mininet> ports
s1 lo:0 s1-eth1:1 s1-eth2:2 s1-eth3:3 s1-eth4:4
s2 lo:0 s2-eth1:1 s2-eth2:2
s3 lo:0 s3-eth1:1 s3-eth2:2
s4 lo:0 s4-eth1:1 s4-eth2:2 s4-eth3:3 s4-eth4:4
mininet> pingall
*** Ping: testing ping reachability
h1 -> X h3 X
h2 -> X X h4
h3 -> h1 X X
h4 -> X h2 X
*** Results: 66% dropped (4/12 received)
mininet> h1 ping -c5 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=12.0 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=13.0 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=12.1 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=11.9 ms
64 bytes from 10.0.0.3: icmp_seq=5 ttl=64 time=11.9 ms

--- 10.0.0.3 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 11.857/12.154/12.953/0.405 ms
mininet> h2 ping -c5 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=23.4 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=23.6 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=23.4 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=24.0 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=24.8 ms

--- 10.0.0.4 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 23.378/23.844/24.768/0.522 ms
```

```
mininet> h1 arp -n
Address HWtype HWaddress Flags Mask Iface
10.0.0.2 (incomplete) h1-eth0
10.0.0.4 (incomplete) h1-eth0
10.0.0.3 ether 00:00:00:00:00:03 C h1-eth0
mininet>
```

(a) Porte, isolamento e ritardi

(b) Cache ARP di h1: risolto solo il partner di slice

Figura 2: Verifica dell'isolamento. `pingall` restituisce 66% dropped (4/12), con la matrice attesa.

La cache ARP di h1 mostra il punto piu' interessante: 10.0.0.3 e' risolto, mentre 10.0.0.2 e 10.0.0.4 restano (*incomplete*). L'isolamento agisce quindi **gia' a livello ARP** e non sul traffico IP: la ARP request di h1 segue il percorso dello slice upper e non raggiunge mai h2 e h4, che percio' non rispondono.

Misura	Upper (h1–h3)	Lower (h2–h4)	Atteso
RTT medio (ping)	12,2 ms	23,8 ms	rapporto $\approx$ 1:2
Throughput (iperf TCP)	9,55 Mbps	1,21 Mbps	rapporto $\approx$ 10:1
Cattura sui link core	solo h1↔h3	solo h2↔h4	separazione totale

```
[SWITCH] s2 connesso
[SWITCH] s3 connesso
[FLOW] s1 [UPPER] 00:00:00:00:01 -> 00:00:00:00:03 : porta 1 -> 3
[FLOW] s2 [UPPER] 00:00:00:00:01 -> 00:00:00:00:03 : porta 1 -> 2
[FLOW] s4 [UPPER] 00:00:00:00:01 -> 00:00:00:00:03 : porta 3 -> 1
[FLOW] s4 [UPPER] 00:00:00:00:03 -> 00:00:00:00:01 : porta 1 -> 3
[FLOW] s2 [UPPER] 00:00:00:00:03 -> 00:00:00:00:01 : porta 2 -> 1
[FLOW] s1 [UPPER] 00:00:00:00:03 -> 00:00:00:00:01 : porta 3 -> 1
[FLOW] s4 [LOWER] 00:00:00:00:04 -> 00:00:00:00:02 : porta 2 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:04 -> 00:00:00:00:02 : porta 2 -> 1
[FLOW] s1 [LOWER] 00:00:00:00:04 -> 00:00:00:00:02 : porta 4 -> 2
[FLOW] s1 [LOWER] 00:00:00:00:02 -> 00:00:00:00:04 : porta 2 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:02 -> 00:00:00:00:04 : porta 1 -> 2
[FLOW] s4 [LOWER] 00:00:00:00:02 -> 00:00:00:00:04 : porta 4 -> 2

mininet> h1 ping -c10 h3 &
mininet> h2 ping -c10 h4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=64 time=53.8 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=64 time=23.8 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=64 time=23.6 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=64 time=23.2 ms
64 bytes from 10.0.0.4: icmp_seq=5 ttl=64 time=23.8 ms
64 bytes from 10.0.0.4: icmp_seq=6 ttl=64 time=23.5 ms
64 bytes from 10.0.0.4: icmp_seq=7 ttl=64 time=24.8 ms
64 bytes from 10.0.0.4: icmp_seq=8 ttl=64 time=23.2 ms
64 bytes from 10.0.0.4: icmp_seq=9 ttl=64 time=23.1 ms
64 bytes from 10.0.0.4: icmp_seq=10 ttl=64 time=23.8 ms

--- 10.0.0.4 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9012ms
rtt min/avg/max/mdev = 23.145/26.655/53.817/9.065 ms
mininet>
```

Figura 3: Catture simultanee su s2-eth1 (sinistra) e s3-eth1 (destra) durante due ping in parallelo. I timestamp si sovrappongono nell’intervallo 13:18:25–13:18:28: i due flussi coesistono sulla stessa rete e ciascuno usa esclusivamente il proprio link core.

5 Service slicing: risultati

Tutti gli host si raggiungono (pingall: 0% dropped), ma il percorso dipende dal servizio: il traffico UDP verso la porta 9999, convenzionalmente “video”, usa lo slice upper da 10 Mbps, ogni altro traffico il lower da 1 Mbps. La classificazione occupa una sola riga di politica, slice_name = ‘UPPER’ if is_video else ‘LOWER’; tutto il resto del metodo e’ calcolo del percorso.

Il test finale usa la stessa coppia di host e lo stesso bitrate, variando solo la porta UDP di destinazione:

Flusso h1 → h3	Slice	Ricevuto	Perdita	Jitter
UDP porta 9999 (video)	UPPER	9,72 Mbps	53%	0,43 ms
UDP porta 5001 (dati)	LOWER	0,97 Mbps	95%	1,64 ms

```
[SWITCH] s1 connesso
[DROP] s1: sorgente sconosciuta fa:e7:95:dc:bc:7b
[SWITCH] s2 connesso
[SWITCH] s3 connesso
[DROP] s2: sorgente sconosciuta 12:65:70:11:aa:38
[SWITCH] s4 connesso
[DROP] s3: sorgente sconosciuta 56:1b:ff:54:57:fb
[DROP] s3: sorgente sconosciuta 82:20:67:fc:0d:40
[DROP] s2: sorgente sconosciuta a6:bc:1e:a2:34:44
[DROP] s4: sorgente sconosciuta 36:de:01:02:92:56
[SWITCH] s4 connesso
[FLOW] s4 [LOWER] 00:00:00:00:03 -> 00:00:00:00:01 : porta 1 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:03 -> 00:00:00:00:01 : porta 2 -> 1
[FLOW] s1 [LOWER] 00:00:00:00:03 -> 00:00:00:00:01 : porta 4 -> 1
[FLOW] s1 [UPPER VIDEO] 00:00:00:00:01 -> 00:00:00:00:03 : porta 1 -> 3
[FLOW] s2 [UPPER VIDEO] 00:00:00:00:01 -> 00:00:00:00:03 : porta 1 -> 2
[FLOW] s4 [UPPER VIDEO] 00:00:00:00:01 -> 00:00:00:00:03 : porta 3 -> 1
[FLOW] s4 [LOWER] 00:00:00:00:03 -> 00:00:00:00:01 : porta 1 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:03 -> 00:00:00:00:01 : porta 2 -> 1
[FLOW] s1 [LOWER] 00:00:00:00:03 -> 00:00:00:00:01 : porta 4 -> 1
[FLOW] s1 [LOWER] 00:00:00:00:01 -> 00:00:00:00:03 : porta 1 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:01 -> 00:00:00:00:03 : porta 1 -> 2
[FLOW] s4 [LOWER] 00:00:00:00:01 -> 00:00:00:00:03 : porta 4 -> 1
[FLOW] s1 [LOWER] 00:00:00:00:01 -> 00:00:00:00:03 : porta 1 -> 4
[FLOW] s3 [LOWER] 00:00:00:00:01 -> 00:00:00:00:03 : porta 1 -> 2
[FLOW] s4 [LOWER] 00:00:00:00:01 -> 00:00:00:00:03 : porta 4 -> 1

mininet> h3 iperf -s -u -p 9999 &
mininet> h1 iperf -c 10.0.0.3 -u -p 9999 -b 20M -t 10 -i 1
-----
Client connecting to 10.0.0.3, UDP port 9999
Sending 1470 byte datagrams, IPG target: 560.76 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 48443 connected with 10.0.0.3 port 9999
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-1.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 1.0000-2.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 2.0000-3.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 3.0000-4.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 4.0000-5.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 5.0000-6.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 6.0000-7.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 7.0000-8.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 8.0000-9.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 9.0000-10.0000 sec 2.50 MBytes  21.0 Mbits/sec
[ 1] 0.0000-10.0003 sec 25.0 MBytes  21.0 Mbits/sec
[ 1] Sent 17836 datagrams
[ ID] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter      Lost/Total Datagrams
[ 1] 0.0000-10.0849 sec 11.7 MBytes  9.72 Mbits/sec  0.433 ms 9580/17836 (53%)
[ 1] 0.0000-10.0849 sec 58 datagrams received out-of-order
mininet>
```

Figura 4: Log del controller in modalita’ service e risultato del flusso video: tre righe [UPPER VIDEO] (s1, s2, s4) e 9,72 Mbps ricevuti.


```

mininet> h3 iperf -s -u -p 5001 &
mininet> h1 iperf -c 10.0.0.3 -u -p 5001 -b 20M -t 10 -i 1
-----
Client connecting to 10.0.0.3, UDP port 5001
Sending 1470 byte datagrams, IPG target: 560.76 us (kalman adjust)
UDP buffer size: 208 KByte (default)
-----
[ 1] local 10.0.0.1 port 53491 connected with 10.0.0.3 port 5001
[ ID] Interval      Transfer      Bandwidth
[ 1] 0.0000-1.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 1.0000-2.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 2.0000-3.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 3.0000-4.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 4.0000-5.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 5.0000-6.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 6.0000-7.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 7.0000-8.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 8.0000-9.0000 sec  2.50 MBytes  21.0 Mbits/sec
[ 1] 9.0000-10.0000 sec 2.50 MBytes  21.0 Mbits/sec
[ 1] 0.0000-10.0010 sec 25.0 MBytes  21.0 Mbits/sec
[ 1] Sent 17837 datagrams
[ 1] Server Report:
[ ID] Interval      Transfer      Bandwidth      Jitter    Lost/Total Datagrams
[ 1] 0.0000-11.2007 sec 1.30 MBytes   973 Kbits/sec  1.642 ms 16909/17836 (95%)
mininet>

```

Figura 5: Stesso flusso verso la porta 5001: 973 Kbit/s con 95% di perdita.

Instradamento asimmetrico. Nei log il traffico di ritorno $h3 \rightarrow h1$ viaggia sul lower anche durante il test video. La classificazione esamina la porta UDP di *destinazione*, e i pacchetti di ritorno sono diretti alla porta effimera del client. Per uno streaming reale e' il comportamento desiderabile, poiche' il flusso dati e' unidirezionale.

6 Granularita' delle flow entry

Se le regole di inoltro si basassero unicamente sugli indirizzi MAC (livello 2), si creerebbero problemi nella classificazione del traffico. Basterebbe infatti un semplice *ping* iniziale per generare una regola troppo generica, capace di catturare automaticamente tutto il traffico successivo tra due host, compreso lo streaming video.

In questo scenario, i pacchetti video non arriverebbero mai al controller per essere analizzati e smistati. Saltando i controlli, il video verrebbe dirottato nello *slice* sbagliato e più lento (causando un crollo delle prestazioni, ad esempio da 9,7 Mbps a 974 Kbit/s).

Per evitare questo problema, il controller è programmato per creare regole "su misura". Ogni regola generata è specifica per il tipo esatto di traffico che deve gestire, garantendo che la connessione base e il flusso video vengano sempre separati e instradati correttamente.

```

if ip4 is None:
    match = parser.OFPMatch(in_port=in_port, eth_src=src, eth_dst=dst,
                             eth_type=eth.ethertype)
elif ip4.proto == in_proto.IPPROTO_UDP:
    match = parser.OFPMatch(in_port=in_port, eth_src=src, eth_dst=dst,
                             eth_type=ether_types.ETH_TYPE_IP,
                             ip_proto=in_proto.IPPROTO_UDP,
                             udp_dst=udp_hdr.dst_port)
    priority = PRIO_VIDEO if is_video else PRIO_FORWARD
else:
    match = parser.OFPMatch(in_port=in_port, eth_src=src, eth_dst=dst,
                             eth_type=ether_types.ETH_TYPE_IP,
                             ip_proto=ip4.proto)

```

Per fare match su `udp_dst` OpenFlow richiede che siano specificati anche `eth_type` e `ip_proto`: i prerequisiti servono allo switch per sapere come interpretare i byte in cui cercare la porta. Il risultato e' verificabile direttamente nella flow table.

```

ncis@ncis-vm:~/Desktop/ncis$ sudo ovs-ofctl dump-flows s1 -O OpenFlow13
cookie=0x0, duration=14.344s, table=0, n_packets=6262, n_bytes=9468144, idle_timeout=30, priority=20,udp,in_port="s1-eth1",dl_src=00:00:00:00:01,dl_dst=00:00:00:00:00:03,tp_dst=9999 actions=output:"s1-eth3"
cookie=0x0, duration=7.328s, table=0, n_packets=3134, n_bytes=4738608, idle_timeout=30, priority=10,udp,in_port="s1-eth1",dl_src=00:00:00:00:01,dl_dst=00:00:00:00:00:03,tp_dst=5001 actions=output:"s1-eth4"
cookie=0x0, duration=8.843s, table=0, n_packets=0, n_bytes=0, idle_timeout=30, priority=10,arp,in_port="s1-eth1",dl_src=00:00:00:00:00:01,dl_dst=00:00:00:00:00:03 actions=output:"s1-eth4"
cookie=0x0, duration=8.815s, table=0, n_packets=0, n_bytes=0, idle_timeout=30, priority=10,arp,in_port="s1-eth4",dl_src=00:00:00:00:00:03,dl_dst=00:00:00:00:00:01 actions=output:"s1-eth1"
cookie=0x0, duration=1066.247s, table=0, n_packets=62, n_bytes=13286, priority=0 actions=CONTROLLER:65535

```

Figura 6: dump-flows su s1 con due flussi UDP attivi. Le prime due regole hanno identici `in_port`, `dl_src` e `dl_dst`: cambia solo `tp_dst` (9999 contro 5001), e con esso priorit  (20 contro 10) e porta di uscita (`s1-eth3` contro `s1-eth4`).

7 Il traffico broadcast e l'assenza di flooding

Anche se i percorsi sono rigidamente vincolati, il traffico ARP riesce comunque ad attraversare la rete. In modalit  *topology*, infatti, lo *slice* di un pacchetto viene sempre determinato dall'indirizzo MAC sorgente, indipendentemente dal fatto che la destinazione sia o meno un indirizzo di gruppo. Di conseguenza, anche il traffico *broadcast* consulta la tabella di instradamento `SLICE_PATH` come qualsiasi altro pacchetto, seguendo cos  il cammino predefinito senza violare le regole di isolamento:

Switch	Entra da	Tabella consultata	Esce da	Verso
s1	porta 1	SLICE_PATH['UPPER'][1] = {1:3, 3:1}	porta 3	s2
s2	porta 1	SLICE_PATH['UPPER'][2] = {1:2, 2:1}	porta 2	s4
s4	porta 3	SLICE_PATH['UPPER'][4] = {1:3, 3:1}	porta 1	h3

Come illustrato nella tabella, il traffico ignora sistematicamente le interfacce esterne allo *slice* di origine. Questo comportamento spiega il motivo per cui la risoluzione ARP avviene in modo isolato, coinvolgendo solo gli host autorizzati.

L'aspetto fondamentale di questo meccanismo è la totale assenza di *flooding*: il pacchetto *broadcast* viene inoltrato sempre e solo su un'unica porta. Questa logica è efficace perché ogni *slice* collega due soli host lungo un percorso lineare; di conseguenza, per raggiungere l'altro capo della rete basta seguire la direzione opposta a quella di arrivo, una regola imposta dalla mappa SLICE_PATH. Eliminando alla radice la replicazione dei pacchetti, l'anello fisico della topologia risulta ininfluente e i loop di rete vengono prevenuti per costruzione.

È importante notare che per il traffico di gruppo il rigido controllo di isolamento viene saltato. Un indirizzo di *broadcast* come `ff:ff:ff:ff:ff:ff`, infatti, non appartiene ad alcuno *slice*. Tuttavia, il sistema mantiene il controllo assoluto smistando i pacchetti in base alla loro destinazione, secondo tre scenari precisi ed esaustivi:

Destinazione	Classe	Trattamento
Broadcast o multicast	BUM	Segue il percorso dello slice del mittente
Unicast, stesso slice	unicast	Inoltro e installazione della flow entry
Unicast, altro slice	unicast	Regola di scarto (priorità 5)

Come illustrato nello schema, il traffico di gruppo viene gestito direttamente dal controller senza generare alcuna *flow entry* sugli switch. Installare regole fisse per il *broadcast* occuperebbe infatti spazio inutile nelle tabelle di inoltro per pochissimi pacchetti, considerando che la cache ARP degli host memorizza le risoluzioni per diversi minuti.

Questa soluzione di inoltro lineare funziona perché ogni *slice* collega esattamente due host. Di conseguenza, per instradare i pacchetti basta una semplice mappa "uno-a-uno" del tipo `{porta: porta}`.

Al contrario, nella modalità *service* il traffico *broadcast* deve raggiungere contemporaneamente tutti gli host della rete, richiedendo la replicazione dei pacchetti su più porte dello stesso switch. Per gestire questa logica "uno-a-molti", si utilizza la struttura BCAST_TREE: una mappa del tipo `{porta: [lista di porte]}` che instrada in modo sicuro il *flooding* lungo un albero statico predefinito.

8 Altre scelte di progetto ed esecuzione

Installazioni ridondanti sul canale di controllo. Durante i primi test lo stesso FlowMod veniva installato cinque o sei volte per switch a ogni nuovo flusso. La causa è una finestra temporale: fra l'istante in cui il controller decide e quello in cui la regola diventa attiva sullo switch, i pacchetti successivi sono già in volo verso il controller e generano ciascuno una richiesta identica. Su un flusso da 20 Mbps questo significa decine di messaggi inutili sulla connessione di controllo. OpenFlow prevede per questo il campo `buffer_id`, che consente di installare la regola e sbloccare il pacchetto in attesa con un unico messaggio, e il codice lo implementa; ma Open vSwitch **non implementa il buffering dei pacchetti** e invia sempre il frame completo con `buffer_id = OFP_NO_BUFFER`, quindi quel ramo non viene mai eseguito. La soluzione è stata una cache lato controller (`recent_flows`) che riconosce le richieste già soddisfatte di recente e si limita a inoltrare il pacchetto: le installazioni ridondanti sono scese a una per switch.

Misure in UDP. Le misure definitive del service slicing sono in UDP perché iperf in TCP conta i byte alla consegna al socket e non alla ricezione: la coda assorbe l'eccesso, i valori istantanei superano la banda nominale e il test sfiora la durata richiesta (*bufferbloat*). In UDP il Server Report riporta il rate effettivamente ricevuto. Le flow entry hanno un `idle_timeout` di 30 s, che limita la crescita delle tabelle e fa sì che una modifica alla configurazione abbia effetto in tempi brevi senza riavviare il controller.

Terminale 1 — controller	Terminale 2 — rete
<code>source ~/ryu-venv/bin/activate</code>	<code>sudo mn -c</code>
<code>ryu-manager slicing_controller.py</code>	<code>sudo python3 topology.py</code>
<i>(o con SLICING_MODE=service)</i>	<code>mininet> pingall</code>

Gli screenshot completi di tutti gli esperimenti, compresi i test TCP e le verifiche di configurazione, sono nel repository sotto `docs/screenshots/`; il README.md riporta la procedura di installazione e i vincoli sulle versioni.